

```

title: DataSafe Room – Propuesta de alineación con IDSA Dataspace Protocol
project: datasafe-room
status: internal-proposal
updated: 2026-05-07
owner: Black Tower Consulting / Ventura
scope: cambios concretos para especificación técnica sin sobrecomplicar MVP

```

DataSafe Room — Propuesta de alineación con IDSA Dataspace Protocol

0. Decisión recomendada

Mantener la decisión base de **monolito modular + workers + storage privado + Controlled Export First**. La alineación con IDSA Dataspace Protocol debe incorporarse como **target de interoperabilidad y modelo canónico**, no como federación completa en el MVP.

Cambio de posicionamiento técnico:

- **MVP/P1:** DSP-ready by design.
- Modelar y persistir metadatos suficientes para proyectar Data Products a **DCAT Catalog/Dataset/Distribution**.
- Modelar políticas internas con una proyección **ODRL JSON-LD limitada**.
- Mapear `OutputRequest` + `Agreement` + `ExportContract` a un **shadow model** de Contract Negotiation.
- Mapear descarga/export aprobado a un **shadow model** de Transfer Process por HTTPS.
- Exponer un endpoint de versión `/well-known/dspace-version` cuando se habilite el modo interoperabilidad.
- **No MVP:** no desplegar EDC, no catálogo federado, no negociación remota real multi-conector, no verifiable credentials obligatorias, no conectores ERP/MES/PLM.
- **V1/V2:** activar façade DSP por HTTPS y, si hay demanda real, integrar con EDC u otro connector.

1. Cambios de arquitectura

1.1 Añadir módulo: Dataspace Interoperability Adapter

Agregar un bounded context nuevo, pero implementado dentro del monolito modular:

Dataspace Interoperability Adapter

Responsabilidades:

- Publicar metadatos de versión del protocolo vía `GET /well-known/dspace-version`.
- Proyectar `Room` + `DataProductVersion` + `Asset` a **DCAT Catalog/Dataset/Distribution**.
- Proyectar `Policy` + `Agreement` + `ExportContract` a **ODRL Policy/Offer/Agreement**.
- Mapear estados internos a **Contract Negotiation** y **Transfer Process**.
- Exponer, detrás de feature flag, endpoints HTTPS compatibles con DSP:

- Catalog: `/catalog/request` , `/catalog/datasets/{id}` .
- Contract Negotiation: `/negotiations/...` .
- Transfer Process: `/transfers/...` .
- Validar que ninguna proyección publique datos crudos, URLs S3 internas, PII o secretos no aprobados.

Componentes concretos:

- `DspaceVersionController` .
- `DcatCatalogProjectionService` .
- `OdrPolicyProjectionService` .
- `DspContractNegotiationMapper` .
- `DspTransferProcessMapper` .
- `InteropPublicationRepository` .
- `InteropConformanceTests` .

Dependencias:

- Lee desde `Room Service` , `Data Product Registry` , `Policy and Agreement Service` , `Output Request Service` , `Audit Service` .
- No decide permisos por sí mismo: siempre invoca `Policy Engine` .
- No accede directo a object storage: obtiene artefactos autorizados mediante `Asset and Evidence Service` .

No-go explícitos:

- No meter un EDC Connector dentro del MVP.
- No abrir catálogo público por defecto.
- No hacer transfer automático a terceros sin aprobación humana.
- No prometer "IDSA/Gaia-X/EDC compliant"; usar "DSP-aligned / DSP-ready".

1.2 Cambiar el diagrama C4

Añadir un contenedor opcional junto al API:

```
External Dataspace Participant / Connector
-> HTTPS DSP Facade / Interop Adapter
-> API + Policy Engine
-> Catalog/Policy/Workflow/Storage
```

Notas para el diagrama:

- Etiquetar como **optional / feature-flagged**.
- Mostrar `/well-known/dspace-version` como endpoint de discovery.
- Mostrar `catalog` , `negotiations` y `transfers` como façade, no como servicios core separados.

2. Cambios de entidades y modelo de datos

2.1 Mantener entidades actuales, añadir campos de interoperabilidad

No crear un modelo dataspace paralelo completo. Añadir identificadores y snapshots JSON-LD sobre entidades existentes.

organizations

Añadir:

```
- dataspace_participant_id text null
- did_web text null
- legal_registration_number text null
- interop_metadata jsonb null
```

Uso:

- Identificar provider/consumer/auditor en proyecciones DSP.
- Preparar DID/VC en V2 sin exigirlo en MVP.

rooms

Añadir:

```
- catalog_visibility enum(private, room_participants, interop_enabled) default private
- interop_enabled boolean default false
- interop_base_url text null
```

Uso:

- Controlar si una sala puede proyectarse a catálogo DSP.
- Evitar que el MVP publique catálogos por accidente.

data_products

Añadir:

```
- dcat_dataset_id text null
- dcat_keywords jsonb null
- dcat_theme text null
- publisher_organization_id uuid fk null
- rights_summary text null
- license_url text null
- contact_point jsonb null
```

Mapeo:

- DataSafe DataProduct → DCAT Dataset lógico.
- owning_organization_id → DCAT publisher.
- rights_summary / license_url → restricciones humanas visibles.

data_product_versions

Añadir:

```
- dcat_version_iri text null
- dcat_jsonld_snapshot jsonb null
- schema_distribution_jsonld jsonb null
- dsp_asset_id text null
```

Mapecto:

- `DataProductVersion` publicado → versión concreta del DCAT Dataset.
- `schema_distribution_jsonld` describe formato, campos, periodo, MIME y checksum de manifest, no el dataset crudo.

assets

Añadir:

```
- dcat_distribution_id text null
- media_type text null
- conforms_to text null
- access_service text null
- access_url_public boolean default false
```

Regla:

- En MVP, `access_url_public=false`; el catálogo no debe contener URLs de descarga directa.
- La distribución apunta al proceso de solicitud/export, no al objeto S3/MinIO.

policies

Añadir:

```
- odr_policy_id text null
- odr_profile text null
- odr_jsonld_snapshot jsonb null
- odr_supported_actions jsonb null
```

ODRL subset recomendado para MVP:

- Permissions: uso para finalidad aprobada, lectura de metadatos, export aprobado.
- Prohibitions: redistribución, uso competitivo/no declarado, ingeniería inversa si aplica, export de campos bloqueados.
- Duties: retención/borrado, mantener atribución, respetar watermark si V1.
- Constraints: finalidad, duración, organización destinataria, clasificación, campo, formato, jurisdicción si aplica.

agreements

Añadir:

```
- odr_agreement_id text null
- contract_agreement_id text null
```

```
- odrl_agreement_jsonld_snapshot jsonb null
- dsp_negotiation_id text null
```

Mapeo:

- Agreement humano/legal aceptado → ContractAgreement /ODRL agreement snapshot.
- No sustituye contrato legal; lo representa machine-readable para interoperabilidad.

output_requests

Añadir:

```
- dsp_negotiation_id text null
- dsp_negotiation_role enum(provider, consumer) null
- dsp_negotiation_state text null
- counterparty_participant_id text null
- contract_offer_snapshot jsonb null
```

Mapeo interno:

- draft/submitted → negociación inicial/request.
- approval_pending → oferta/contraoferta pendiente internamente.
- approved → acuerdo finalizable.
- rejected/revoked/expired → terminated.

export_contracts

Añadir:

```
- contract_agreement_id text null
- odrl_policy_snapshot jsonb null
- dsp_transfer_process_id text null
- transfer_type enum(https_signed_url, sftp, private_bucket, manual_pack) default
https_signed_url
- transfer_state text null
```

Mapeo:

- ExportContract aprobado → contrato de uso y base para transferencia.
- download-url aprobado → Transfer Process HTTPS controlado.

2.2 Nueva tabla ligera: interop_publications

Agregar una sola tabla genérica para no duplicar dominio:

```
interop_publications
- id uuid pk
- room_id uuid fk null
- entity_type enum(room, data_product, data_product_version, policy, agreement,
output_request, export_contract, asset)
- entity_id uuid
```

```
- protocol enum(dsp, dcat, odr1)
- protocol_version text
- external_id text
- jsonld_snapshot jsonb
- visibility enum(private, room_participants, external)
- status enum(draft, published, revoked, superseded)
- published_at timestamptz null
- revoked_at timestamptz null
- created_at timestamptz
- updated_at timestamptz
```

Uso:

- Cachear snapshots JSON-LD reproducibles.
- Auditar qué se publicó o proyectó.
- Evitar recalcular catálogo y policy en cada request.

3. Cambios de APIs

3.1 API interna REST existente: añadir endpoints de proyección

Mantener bajo `/api/v1` ; autenticado y room-scoped.

```
GET /api/v1/interop/status
GET /api/v1/rooms/{room_id}/interop/catalog/dcat
POST /api/v1/rooms/{room_id}/interop/catalog/publish
POST /api/v1/rooms/{room_id}/interop/catalog/revoke
GET /api/v1/rooms/{room_id}/data-products/{data_product_id}/versions/{version_id}/dcat
GET /api/v1/rooms/{room_id}/policies/{policy_id}/odr1
POST /api/v1/rooms/{room_id}/policies/{policy_id}/odr1/validate
GET /api/v1/rooms/{room_id}/agreements/{agreement_id}/odr1
GET /api/v1/rooms/{room_id}/output-requests/{request_id}/dsp-negotiation
GET /api/v1/rooms/{room_id}/output-requests/{request_id}/dsp-transfer
```

Regla MVP:

- Estos endpoints pueden existir en P1 como herramientas internas/admin.
- No son garantía de conformidad DSP externa.
- Sirven para validar que el modelo DataSafe no bloquea futura interoperabilidad.

3.2 Well-known endpoint de versión DSP

Añadir cuando `interop_enabled=true` a nivel deployment:

```
GET /.well-known/dspace-version
```

Respuesta recomendada en demo/piloto:

```
{
  "service": "DataSafe Room",
```

```

"mode": "dsp-ready-controlled-export",
"supportedProtocolVersions": [
  {
    "protocol": "dataspace-protocol",
    "version": "2024-1",
    "binding": "https",
    "baseUrl": "https://example.com/api/dsp/2024-1",
    "capabilities": [
      "catalog.request",
      "catalog.dataset.read",
      "odrl.policy.projection",
      "contract.negotiation.shadow",
      "transfer.process.https-controlled-export"
    ],
    "conformance": "partial-profile"
  }
]
}

```

Nota de especificación:

- Validar el schema exacto contra la versión DSP seleccionada antes de declarar interoperabilidad pública.
- En MVP usar `conformance=partial-profile` o equivalente interno; no anunciar “full connector”.

3.3 DSP HTTPS façade opcional

Feature flag: `DSP_FACADE_ENABLED=false` por defecto.

Base path sugerido:

```
/api/dsp/2024-1
```

Endpoints mínimos a planificar, no todos para MVP:

Catalog:

```

POST /api/dsp/2024-1/catalog/request
GET /api/dsp/2024-1/catalog/datasets/{id}

```

Contract Negotiation:

```

GET /api/dsp/2024-1/negotiations/{providerPid}
POST /api/dsp/2024-1/negotiations/request
POST /api/dsp/2024-1/negotiations/{providerPid}/request
POST /api/dsp/2024-1/negotiations/{providerPid}/events
POST /api/dsp/2024-1/negotiations/{providerPid}/agreement/verification
POST /api/dsp/2024-1/negotiations/{providerPid}/termination

```

Transfer Process:

```
GET /api/dsp/2024-1/transfers/{providerPid}
POST /api/dsp/2024-1/transfers/request
POST /api/dsp/2024-1/transfers/{providerPid}/start
POST /api/dsp/2024-1/transfers/{providerPid}/completion
POST /api/dsp/2024-1/transfers/{providerPid}/termination
POST /api/dsp/2024-1/transfers/{providerPid}/suspension
```

MVP/P1 behavior recomendado:

- `catalog/request` : devuelve catálogo room-scoped solo si el caller está autenticado/autorizado.
- `negotiations/request` : crea o referencia un `OutputRequest` , pero no salta aprobación humana.
- `transfers/request` : solo funciona si existe `ExportContract` aprobado; devuelve estado, no URL final hasta `start` y policy check.
- `transfers/start` : genera URL firmada de corta duración usando el flujo existente.
- Todos los eventos se registran en `audit_events` .

3.4 Cambios en export pack

Añadir al ZIP generado:

```
export-{output_request_id}.zip
├─ dataspace/
│   └─ dcat_dataset.jsonld
│   └─ odrl_policy.jsonld
│   └─ contract_agreement.jsonld
│   └─ transfer_process.jsonld
```

En `manifest.json` , añadir:

```
{
  "interop": {
    "dataspace_protocol_target": "2024-1",
    "dcat_dataset_id": "urn:datasafe:dcat:dataset:...",
    "odrl_policy_id": "urn:datasafe:odrl:policy:...",
    "contract_agreement_id": "urn:datasafe:contract:agreement:...",
    "transfer_process_id": "urn:datasafe:transfer:...",
    "conformance": "partial-profile-controlled-export"
  }
}
```

4. Cambios en Policy Engine

4.1 Añadir capa de proyección ODRL, no reemplazar el engine

No cambiar el enforcement MVP a ODRL puro. El engine interno sigue siendo código simple/ABAC. ODRL se usa como representación externa/auditable.

Nuevo flujo:

1. Usuario configura política en UI simple.

2. DataSafe guarda `policies` + `policy_rules` como hoy.
3. `OdrlPolicyProjectionService` genera snapshot ODRL limitado.
4. `policy_evaluation_worker` evalúa reglas internas.
5. `export_builder_worker` incluye `odrl_policy.jsonld` y `policy_snapshot.json`.

Ventaja:

- UX no se complica.
- Enforcement no depende de un motor ODRL maduro desde el día uno.
- Interoperabilidad futura queda preparada.

4.2 ODRL profile mínimo DataSafe

Definir un perfil interno:

```
profile: https://datasafe.room/profiles/odrl/controlled-export/1.0
```

Acciones soportadas:

```
- read_metadata
- view_preview
- use_for_purpose
- export_approved_output
- download_artifact
- retain_until
- delete_after
```

Constraints soportadas:

```
- purpose
- room_id
- organization_id
- recipient_participant_id
- data_product_version_id
- field_name
- classification_level
- expiration_datetime
- max_download_count
- allowed_format
```

Duties soportadas:

```
- keep_attribution
- delete_after_retention
- do_not_redistribute
- preserve_confidentiality
- apply_watermark_when_enabled
```

5. Cambios de roadmap

P0 — Demo técnica

Mantener sin DSP externo. Añadir solo preparación barata:

- IDs estables tipo URN para `room`, `data_product`, `policy`, `agreement`, `output_request`, `export_contract`.
- `manifest.json` con bloque `interop` aunque sea parcial.
- Tests unitarios de generación de DCAT/ODRL snapshots con datos sintéticos.

P1 — MVP piloto real controlado

Añadir a P1:

- Campos de interoperabilidad en DB.
- `interop_publications`.
- Proyecciones DCAT/ODRL autenticadas bajo `/api/v1/.../interop`.
- `/well-known/dspace-version` deshabilitado por defecto y habilitable por deployment.
- Contract Negotiation shadow state sobre `OutputRequest`.
- Transfer Process shadow state sobre `ExportContract` + `download-url`.
- Export pack con carpeta `dataspace/*.jsonld`.

Mantener fuera de P1:

- DSP façade pública.
- EDC Connector.
- Catálogo federado.
- DID/VC obligatorio.

P2 — V1 enterprise / interoperabilidad privada

Añadir:

- DSP HTTPS façade en entorno sandbox/piloto con allowlist de participantes.
- `catalog/request` funcional para participantes autorizados.
- `negotiations/request` que crea `OutputRequest` sin saltar approval.
- `transfers/request/start/completion/termination` mapeado a exports controlados.
- API scoped tokens/mTLS si el piloto lo exige.
- Conformance tests contra fixtures DSP seleccionados.

P3 — Dataspace-ready avanzado

Reformular P3 actual:

- EDC/DSP Connector opcional si hay contraparte real.
- Catálogo federado solo si hay más de un participante externo con necesidad repetida.
- Verifiable credentials/DID para identidad organizacional.
- ODRL más completo o policy engine especializado.
- Conectores ERP/MES/PLM read-only aprobados.
- No pasar a P3 sin threat model y caso comercial validado.

6. Cambios concretos en criterios de aceptación

Añadir al criterio de aceptación del MVP técnico, sin bloquear MVP base:

- Un Data Product publicado puede generar un `dcat_dataset.jsonld` válido contra fixtures internos.
- Una Policy activa puede generar un `odrl_policy.jsonld` con purpose, assignee, target, prohibition y duty mínimos.
- Un Output Request aprobado genera `contract_agreement_id` y snapshot `contract_agreement.jsonld` en el export pack.
- Una descarga aprobada puede representarse como `transfer_process.jsonld` con estado `started/completed/terminated`.
- El endpoint `/.well-known/dspace-version` existe en sandbox si `interop_enabled=true`, y queda deshabilitado en demo pública si no se quiere anunciar capacidad.
- Ningún endpoint de interop expone datos crudos ni URLs internas sin policy check.

7. Cambios en decisiones abiertas

Añadir estas decisiones al final de la especificación:

- **Versión DSP target:** usar IDSA Dataspace Protocol 2024-1 como target inicial, validando cambios antes de implementación pública.
- **Modo interop:** empezar con `partial-profile-controlled-export`, no full connector.
- **Endpoint well-known:** exponer `/.well-known/dspace-version` solo en entornos con interoperabilidad habilitada.
- **Catálogo:** room-scoped y autenticado por defecto; nunca catálogo público global en MVP.
- **ODRL:** proyección desde reglas internas, no motor de enforcement principal en MVP.
- **Transfer:** HTTPS signed URL controlado como primer binding; no data plane federado.

8. Texto corto para insertar en la especificación ejecutiva

Insertar en sección 0, después de “Principio de alcance”:

Principio de interoperabilidad:

DataSafe Room será DSP-ready, no un dataspace federado completo en el MVP. El modelo interno debe poder proyectar Data Products como DCAT, Policies/Agreements como ODRL, Output Requests como Contract Negotiation y exports aprobados como Transfer Process sobre HTTPS. Estas capacidades se implementan primero como snapshots y endpoints internos/autenticados; la façade DSP pública y EDC quedan para V1/V2 si hay contraparte real.

9. Fuentes técnicas usadas para esta propuesta

- IDSA / Eclipse Dataspace Protocol, especificaciones 2024-1:
- Common protocol: endpoint `/.well-known/dspace-version`.
- Catalog HTTPS binding: `/catalog/request`, `/catalog/datasets/{id}`.

- Contract Negotiation HTTPS binding: /negotiations/... .
- Transfer Process HTTPS binding: /transfers/... .
- Especificación técnica actual de DataSafe Room datasafe-room-technical-specification-2026-05-07.md .
- Diagramas UX/arquitectura actuales datasafe-room-technical-diagrams-ux-flow-2026-05-07.md .